ROS 2 - STM32 Communication Process

Mobile Robot Platform

Technical Guide

Document Version	V1.1
Release Date	2025-12-08
Scope	ROS 2 main controller to/from STM32 motion controller

ROS 2 Integration Reference

Document Notes

This guide explains the communication architecture between a ROS 2 main controller and an STM32-based motion controller, including hardware connectivity, Linux device aliases, ROS 2 package organization, node behavior, serial data reception, and velocity-command transmission.

Implementation details such as package, launch-file, node, and device-alias names may vary with the delivered platform configuration. This guide focuses on interfaces, data flow, and integration behavior.

Version History

Version	Date	Description
V1.0	2025-05-20	Initial release
V1.1	2025-12-08	Updated ROS 2 integration details

Contents

1. ROS 2 and STM32 Communication Process

1.1 Hardware Connection between ROS 2 and STM32

1.2 Software Connection and Stable Device Aliases

1.3 ROS 2 Base Communication Package

1.4 Main Bring-up Launch File

1.5 Base Communication Node

1.6 Data Reception, Processing, and Transmission

1. ROS 2 and STM32 Communication Process

A typical mobile robot platform contains two core controllers together with commonly used sensors such as LiDAR, cameras, and IMUs. The two core controllers are the ROS 2 main controller and the STM32 motion controller.

Common ROS 2 main controllers include NVIDIA Jetson development boards such as Jetson Orin Nano and Jetson Orin NX, Raspberry Pi platforms, and other Linux-based embedded computers. STM32F103- and STM32F407-series microcontrollers are commonly used on the low-level controller to acquire odometry and IMU data and to execute motor-control commands.



1.1 Hardware Connection between ROS 2 and STM32

1.1.1 Hardware connection overview

The ROS 2 main controller communicates with the STM32 control board through a USB Type-C data cable. A USB-to-TTL level-conversion chip on the STM32 board converts the USB serial signal to TTL levels that can be processed by the STM32 microcontroller.



After the STM32 controller is connected to the ROS 2 computer, the USB device can be inspected with:

```
lsusb
```

The serial device can then be checked under /dev. Depending on the Linux system and driver, it may appear as /dev/ttyCH343USB0 or /dev/ttyUSB0.

```
ll /dev
```

1.1.2 Assigning the converter serial number

Linux device names can change after reconnecting a USB device. To support stable identification, the CH9102 converter can be assigned a fixed device serial number. In the standard configuration, the STM32 controller uses serial number 0002.

The serial number can be configured in Windows using the CH34xSerCfg.exe utility. After programming the converter, reconnect it to the ROS 2 computer before creating the Linux device alias described in the next section.

1.2 Software Connection and Stable Device Aliases

1.2.1 Creating stable Linux aliases

A robot may connect several USB-to-TTL devices at the same time. In addition to the STM32 controller, LiDAR, inertial-navigation modules, voice modules, or GNSS receivers may also use USB serial converters. To avoid hard-coding changing /dev/tty* device numbers, Linux udev rules can be used to create stable device aliases.

The ROS 2 software stack uses Linux udev rules to create stable aliases for the controller and connected serial devices. A representative device-number mapping is shown below; exact alias names may vary by configuration:

Device	Serial No.	Representative Alias
Serial LiDAR	0001	/dev/robot_lidar
STM32 controller	0002	/dev/robot_controller
External IMU / INS	0003	/dev/robot_imu
Voice sensor	0004	/dev/robot_mic
GNSS receiver	0005	/dev/robot_gnss

After the serial number is configured and the corresponding udev rule is installed, reload the udev rules and reconnect the device:

```
sudo udevadm control --reload-rules
sudo udevadm trigger
```

The STM32 serial interface can then be accessed through a stable controller alias rather than a changing /dev/tty* device name.

```
ll /dev
```

1.3 ROS 2 Base Communication Package

The base communication package establishes the data link between the ROS 2 main controller and the STM32 motion controller. It transfers motion commands from ROS 2 to the STM32 and returns motion/sensor data from the STM32 to ROS 2.

The software stack includes a base communication package that establishes the data link between ROS 2 and the STM32 controller. The representative structure below shows the main components used for ROS 2/STM32 integration.

```
<ros2_workspace>/src/<base_communication_package>
```

Representative package structure:

```
base_communication_package/
├── CMakeLists.txt
├── package.xml
├── include/
├── msg/
├── src/
│   ├── base_communication.cpp
│   └── attitude_solution.cpp
├── config/
│   ├── camera_info.yaml
│   ├── ekf.yaml
│   ├── imu.yaml
│   └── robot_param.yaml
└── launch/
```

- CMakeLists.txt: CMake build script used to define the package name, compilation options, dependencies, and executable nodes.
- package.xml: package metadata, version, description, maintainer information, build dependencies, and runtime dependencies such as rclcpp and std_msgs.
- include/: header files referenced by the source code.
- msg/: custom ROS 2 message definitions.
- src/: C++ source code implementing ROS 2 to/from STM32 communication.
- config/: camera calibration, EKF parameters, IMU filtering parameters, robot-model parameters, and platform/sensor selection settings.
- launch/: launch files used to start the low-level base node and related robot functions.

Launch files and purpose

Launch Component	Function
Base serial launch	Starts the STM32 low-level communication node.
Robot description launch	Loads the URDF, publishes joint states, and broadcasts TF transforms.
Camera launch	Starts the configured camera sensor.
EKF launch	Starts EKF filtering / sensor-fusion nodes.
LiDAR launch	Starts the configured LiDAR sensor.
Sensor bring-up launch	Starts external sensors including LiDAR, camera, and IMU/INS.
Main robot bring-up launch	Starts the core robot functions, including base communication, IMU processing, EKF state estimation, TF, and robot model.

1.4 Main Bring-up Launch File

The main bring-up launch file configures and starts the robot's core ROS 2 functions, including base serial communication, IMU processing, EKF state estimation, TF transforms, and the robot model.

The main bring-up launch file starts the robot's core ROS 2 functions in a coordinated sequence. A representative set of actions is shown below:

Action	Purpose
Robot model selector	Loads the URDF/model configuration for the selected robot variant. Only one platform model should be active at a time.
SLAM / EKF mode selector	Selects the state-estimation configuration according to whether SLAM is active.
Base communication	Starts the low-level serial communication with the STM32 motion controller.
Base link TF	Broadcasts a static TF such as base_footprint → base_link.
IMU TF	Broadcasts the static TF for the gyro/IMU frame.
Joint state publisher	Publishes joint-state information for robot-model visualization.
IMU filter	Starts attitude filtering for IMU orientation estimation.

Action	Purpose
Robot EKF	Starts EKF-based robot-state estimation.

The launch description then adds all declared actions and nodes so that the ROS 2 launcher can execute them as a unified robot bring-up process.

1.5 Base Communication Node

The low-level base launch component starts the base communication node, which is responsible for communication with the STM32 controller. Its implementation includes serial communication, message parsing, state publication, and command handling.

The attitude-solution module uses accelerometer and gyroscope measurements from the STM32-side IMU to calculate the robot's 3D orientation in real time.

The base communication module performs the core serial data exchange between ROS 2 and the STM32 controller. It receives low-level sensor and motion data, publishes ROS 2 topics, subscribes to target velocity commands, and converts those commands into the STM32 serial protocol.

Representative node structure

Initialize → Control Loop → Parse Data → Publish ROS 2 → Callbacks

- Constructor / initialization: opens and configures the serial interface, initializes parameters, publishers, subscribers, and optional services.
- Main control loop: repeatedly acquires STM32 data, processes sensor measurements, calculates odometry and orientation, and services ROS 2 callbacks.
- Data-conversion functions handle IMU conversion, odometry conversion, and checksum verification.
- Velocity-command callback: subscribes to the robot target-velocity topic and transmits velocity commands to the STM32 over the serial link.
- Optional charging/docking callbacks: handle docking velocity, recharge flags, and charging-control service requests on configurations that include automatic charging.

When the base node exits, its destructor sends a zero-velocity command frame before closing the serial port, helping ensure that the robot stops before communication is terminated.

Starting and inspecting the base node

```
ros2 launch <package_name> <main_bringup_launch.py>
```

After launch, the node's publishers, subscribers, services, and message types can be inspected with:

```
ros2 node info <node_name>
```

1.6 Data Reception, Processing, and Transmission

1.6.1 Main control loop

The main control loop repeatedly performs data acquisition, processing, topic publication, and callback execution. A representative cycle is:

1. Read the data frame transmitted by the STM32 controller.
2. Process raw gyroscope and accelerometer data to obtain orientation information.
3. Publish odometry, IMU, and battery-voltage data to ROS 2 topics.
4. If automatic charging is enabled, publish charging state, docking/infrared state, and charging-current information as applicable.
5. Process pending ROS 2 callbacks, including /cmd_vel velocity commands and optional charging/docking commands.

For each iteration, the node records the current ROS 2 time and calculates the sampling interval. This interval is used to integrate velocity into position.

Robot velocity values are corrected using odom_*_scale parameters to account for encoder and gyroscope calibration errors. X and Y represent planar linear velocity, while Z represents yaw rate. Position is then updated by integrating velocity over the sampling interval and compensating the X/Y direction using the current heading.

The resulting velocity and pose estimates are published as odometry. Raw IMU measurements are processed by the attitude-solution module and published as onboard-IMU data. At the end of each loop, the previous timestamp is updated for the next sampling interval.

```
rclcpp::spin_some(this->get_node_base_interface());
```

The non-blocking spin call checks whether callbacks are ready and executes them without stopping the main control loop.

1.6.2 Reading data from the STM32

The serial receive routine reads bytes from the STM32, validates complete frames, and converts the received fields into robot odometry, IMU data, ultrasonic data, and battery-voltage data.

The routine initializes temporary receive variables and buffers, reads data through the configured serial interface, and separates normal motion data from optional auxiliary data where supported.

The frame header and frame tail are extracted during reception. A complete normal data frame is 24 bytes. Before parsing the payload, the function verifies the frame length, frame tail, and checksum.

Read Serial → 24-byte Frame → Frame Check → Checksum → Parse Data → Publish

During payload processing:

- Chassis velocity is parsed first by combining received bytes and converting them into X/Y/Z velocity values in physical units.
- Raw gyroscope and accelerometer values are combined into 16-bit quantities and converted into physical units before publication.
- The standard conversion uses an accelerometer resolution factor of 1671.84 and a gyroscope conversion factor of 0.00026644.
- Battery voltage is reconstructed from its transmitted byte representation and converted back to the required engineering unit.

After all fields pass validation and conversion, the function returns a successful status to the control loop.

1.6.3 Sending velocity commands to the STM32

The /cmd_vel callback subscribes to the robot target-velocity topic and converts the ROS 2 Twist command into the serial protocol used by the STM32 motion controller.

The command object contains target linear and angular velocities. Before transmission, each velocity value is scaled by 1000 and converted to a signed short integer. For example, 0.123 m/s is represented by the scaled integer value 123.

Each signed integer is split into high and low bytes and placed into the transmit buffer. The transmit buffer is 11 bytes long. The checksum is calculated from the first nine bytes and stored in byte 9. Byte 10 is the frame tail, 0x7D.

Byte(s)	Content
0-2	Protocol header / control fields
3-4	X-axis linear velocity: high byte / low byte
5-6	Y-axis linear velocity: high byte / low byte
7-8	Z-axis angular velocity: high byte / low byte

Byte(s)	Content
9	Checksum calculated from bytes 0-8
10	Frame tail: 0x7D

After the frame has been assembled, it is transmitted to the STM32 through the serial interface:

```
serial_port.write(...)
```

/cmd_vel → Scale ×1000 → Pack 16-bit → Checksum → 11-byte Frame → STM32

Quick Interface Summary

The communication architecture described in this guide provides a standard ROS 2 development path from high-level algorithms to the embedded motion controller. The main interfaces are summarized below.

Interface / Component	Role
USB serial link	Physical communication between ROS 2 computer and STM32 controller.
Stable udev alias	Keeps the controller device path consistent across USB reconnections.
Base communication node	Receives STM32 data and publishes ROS 2 state topics; subscribes to velocity commands.
/cmd_vel	Primary target linear/angular velocity command interface.
Odometry publication	Publishes robot velocity and integrated pose information.
IMU publication	Publishes processed orientation and inertial measurements.
URDF / TF	Provides robot model and coordinate-frame relationships.
EKF integration	Supports fused state estimation using odometry and inertial data.
Serial protocol	Converts ROS 2 commands and sensor/state data to compact byte frames for STM32 communication.

This document describes the platform's ROS 2 / embedded-controller integration. The complete software package, source code, firmware, and platform-specific configuration are provided after purchase with the delivered platform.